

# Rapport individuel

Périmètre CI/CD et qualité logicielle

**Auteur**

Amine Messadi

**Client**

Korlea

Mastère DevOps M1, SUP DE VINCI Paris La Défense

Promotion M1DOA 2025-2026

## 1. Pourquoi ce document existe

J'ai rédigé les sections §4.5 (CI/CD) et §4.11 (KPIs) du DAT groupe. Ce rapport ne les reprend pas. Il raconte l'envers de ces sections : comment la chaîne de validation s'est construite, ce qu'elle m'a coûté en mise au point, et les arbitrages que j'ai défendus ou subis. Mon périmètre va du lint au write-back GitOps, c'est-à-dire de la première vérification de syntaxe jusqu'au commit automatique qui déclenche le déploiement. La référence factuelle reste le DAT ; ici, je donne le point de vue de celui qui tenait le pipeline.

## 2. Perspectives d'évolution

Quatre chantiers structurent l'avenir de mon périmètre. Trois prolongent des limites déjà nommées dans le DAT (§2.6.2 et §4.7.8), le quatrième en sort et m'intéresse à titre personnel.

### 2.1 Sortir du digest brut

La référence de release d'une image est aujourd'hui son digest immuable, doublé du tag mutable `:main` que suit l'argocd-image-updater. La mécanique est fiable pour la machine et illisible pour l'humain : l'historique des releases se lit comme une suite de sha de commits, sans repère de version. Je réintroduirais une logique SemVer `v0.x.y` au-dessus du digest, sans la remplacer, via un outil comme *Semantic Release* qui génère le tag et le changelog à partir des messages de commit conventionnels. L'argocd-image-updater préfère nativement la stratégie `semver` ; réintroduire les tags le réaligne sur son mode par défaut, au lieu du contournement que j'ai dû configurer (cf. §3.2). Le numéro de version sert la lecture humaine, le digest reste la garantie d'immuabilité réellement déployée, les deux ne s'excluent pas.

### 2.2 Maîtriser les dépendances

Le DAT note l'absence de Dependabot et de Renovate (§4.7.8). Sur mon périmètre, le besoin ne porte pas que sur les modules Go : il porte aussi sur les versions des actions GitHub, dont les tags mouvants ont cassé mon pipeline (cf. §3.3). Un robot de mise à jour transformerait chaque montée de version, action ou module, en pull request vérifiée par la matrice de validation avant tout merge. La montée de version deviendrait un événement testé, pas une rupture découverte sur un run rouge. Je le configurerais sur les modules Go et sur les actions des deux workflows, avec un groupement des mises à jour mineures pour ne pas noyer l'équipe. La différence avec une veille manuelle reste essentielle : la mise à jour passe par les `required status checks`, donc par la même barrière que tout le reste.

### 2.3 Rotation automatique des secrets par la CI

Les secrets du cluster sont chiffrés via sops-nix, mais leur rotation reste manuelle (§4.12.1). L'angle qui relève de mon périmètre n'est pas la cryptographie des clés, terrain de Tom, mais l'automatisation du geste : un workflow planifié lancerait `sops updatekeys`, ouvrirait une pull request avec le résultat, et laisserait une revue humaine valider avant merge. La rotation deviendrait une routine planifiée et tracée dans Git, chaque occurrence laissant une trace de pull request datée et attribuée, au lieu d'une tâche ponctuelle qu'on repousse jusqu'à l'oublier.

### 2.4 Hors DAT : signer les images et garder la porte du cluster

Le scan Trivy bloque aujourd'hui les CVE HIGH et CRITICAL à la publication (§4.5.2) : c'est une barrière sur la vulnérabilité, pas sur l'origine. Rien ne garantit, côté cluster, qu'une image déployée vient bien de notre pipeline. Je signerais les images en CI avec *Cosign* (projet Sigstore), puis j'ajouterais un contrôleur d'admission *Kyverno* dans K3s qui refuserait tout conteneur non

signé par notre clé. La signature se pose au push GHCR, la vérification se pose à l'admission du Pod : la paire fermerait la chaîne entre le build et le runtime, là où le scan Trivy seul ne couvre qu'une extrémité.

## 2.5 Une limite qui conditionne le passage à l'échelle

L'argocd-image-updater interroge GHCR en continu pour détecter les nouveaux digests. Sur notre MVP, cluster privé à trois nœuds et faible volume, je n'ai jamais déclenché de rate-limiting GHCR (réponse 429 Too Many Requests). À l'échelle d'un Korlea réel, avec des dizaines de microservices surveillés, le comportement du reconciler face à ce rate-limiting devient une limite à anticiper (polling, cache des digests, répartition des requêtes). Je la cite comme un point à surveiller avant une montée en charge, pas comme un incident vécu.

## 3. Analyse critique des limites techniques rencontrées

### 3.1 Le shift-left forcé : quand kube-score et kubeconform rejettent tout

Au milieu du Sprint 2, le pipeline de validation infrastructure est passé d'un état confortable à un mur rouge. Tant que `.github/workflows/validate.yml` ne croisait que des fichiers de configuration basiques, tout passait. Le jour où Karim a poussé ses premiers manifests applicatifs complets (déploiements API et Worker) et où Tom a posé sa structure d'infrastructure Kubernetes, kubeconform et kube-score ont rejeté l'ensemble : le mode `-strict` refuse les champs inconnus, et kube-score applique des règles de sécurité que des manifests fraîchement écrits ne respectent jamais du premier coup.

Il s'est écoulé quatre à cinq jours avant un `validate.yml` qui passait systématiquement. Ce délai n'a bloqué personne : les validations n'étaient pas encore déclarées comme `required status checks` obligatoires, donc Karim et Tom ont continué à pousser pendant que je calais les règles dans ma branche. Ce détail de chronologie compte : la barrière bloquante est venue après la mise au point, pas avant, sans quoi mon apprentissage serait devenu un goulot d'étranglement pour l'équipe.

L'épisode m'a forcé à lire les specs d'API Kubernetes au plus tôt, au lieu de découvrir leurs contraintes au déploiement : la définition concrète du *shift left*. Une partie du calage a porté sur des faux positifs de composants utilitaires. Le cas le plus net reste le conteneur `bitnami/kubectl` du CronJob de purge RGPD, qui doit écrire dans `~/.kube/cache` au runtime, en conflit avec la règle de système de fichiers racine en lecture seule. La solution n'a pas été d'affaiblir la règle globale, mais de poser quatre exclusions ciblées, chacune documentée en commentaire, pour distinguer un vrai défaut de sécurité d'une contrainte d'outil légitime.

### 3.2 Le compromis SemVer et le détour par l'Image Updater

L'absence de SemVer pour le MVP n'est pas une décision que j'aurais prise seul. Elle vient d'une contrainte de vélocité collective : Karim et Tom voulaient voir leurs modifications se refléter sur le cluster sans gérer manuellement des tags Git à chaque commit. Je l'ai accepté à contre-cœur, parce qu'en tant que responsable de la chaîne je savais que ça alourdirait le suivi des versions. La frustration est devenue assumée quand j'ai mesuré que ce compromis me forçait à complexifier mon propre périmètre.

Adapter l'argocd-image-updater pour fonctionner sans SemVer a été un vrai détour technique, pas une ligne de configuration. L'outil préfère nativement suivre des tags SemVer ; pour le faire suivre la combinaison tag `:main` plus pinning par digest, j'ai contourné son comportement par défaut. J'ai passé près de deux jours à tester les bonnes annotations (`image-list`, `write-back-method: git`) et à déboguer les droits de push du token pour que les commits

automatiques aboutissent sur `main`. Le point dur n'était pas la stratégie de mise à jour, mais l'autorisation : un write-back qui échoue silencieusement laisse le cluster sur une version périmée sans rien signaler.

Cet incident touche le même point que celui que Karim raconte de son côté, celui où la CI est verte mais le cluster ne porte pas la bonne version. Lui le vit côté Pod, en `ImagePullBackOff`. Moi je le vis une marche plus haut, quand l'updater doit écrire le nouveau digest dans Git. Quand le write-back ne part pas, rien en aval ne peut converger : c'est pourquoi mon diagnostic commence toujours par l'historique Git du dépôt infra, pas par le cluster.

### 3.3 Le taux de succès à 53 % et sa lecture honnête

Ma faiblesse la plus mesurable porte un chiffre : un taux de succès d'environ 53 % sur les quinze premiers runs analysés de *build.yml* (§4.11.1). J'ai sous-estimé le temps d'apprentissage des validateurs et des actions tierces, et le pipeline applicatif l'a payé au démarrage. Je ne cherche pas à habiller ce chiffre, mais à dire ce qu'il recouvre.

Ces quinze runs tombent en pleine bascule de deux actions que je n'avais pas encore figées : `golangci-lint-action`, qui a changé de schéma en passant en v2, et l'action Trivy, qui a changé d'interface. Tant que ces deux briques bougeaient sous mes pieds, les échecs s'accumulaient sans que mon code soit en cause. La stabilisation s'est faite début Sprint 3, mi-mai, avec quatre derniers runs parfaitement verts. Deux facteurs ont convergé : le calage final des exclusions kube-score (en particulier pour le CronJob de purge) et le pinning des versions d'actions, car des tags mouvants cassaient le pipeline dès qu'un linter mettait à jour ses règles. Je me souviens du run de bascule : la pull request validant le contournement du SecurityContext du conteneur *kubectl*. Après ce run, la CI n'a plus jamais rechuté.

La leçon n'est pas « le taux était biaisé », mais qu'un pipeline dépendant de tags mouvants n'est pas reproductible, et qu'un taux de succès n'a de sens qu'une fois les versions figées. C'est exactement ce qui motive le chantier Dependabot de la partie précédente.

### 3.4 Compromis assumés et ce que je ferais autrement

Ma force tient en une règle que j'ai défendue sans concession : aucun merge sur `main` sans une CI entièrement verte. J'ai déclaré les jobs `lint`, `test` et `build` côté application, plus les six jobs côté infrastructure, comme `required status checks`. Concrètement, personne ne peut merger une branche dont un seul job est rouge, quelle que soit l'urgence. Cette barrière a tenu le cluster à l'écart d'erreurs de syntaxe et de défauts de sécurité.

Ce que je referais autrement tient en une décision : mettre en place Semantic Release dès le premier jour. Le suivi et le rollback fondés sur des digests bruts ont rendu l'historique des releases difficile à lire, et j'ai porté ce coût tout le projet (cf. §2.1).

Un dernier point relève de la lucidité sur l'interdépendance. Mon optimisation de cache n'aurait rien donné sans le Dockerfile de Karim, qui a ordonné les couches en plaçant la résolution des dépendances Go avant la copie du code source. Sans cet ordonnancement, mon cache en mode `max` aurait sauté à chaque modification de code, et le gain observé (d'environ 4 à 5 minutes à un peu plus d'une minute sur le build de l'API) n'aurait jamais existé. Le cache CI est un travail à deux.

## 4. Annexes

### 4.1 Documentation utilisateur : opérer ArgoCD Image Updater

Cette annexe documente le sous-périmètre que je connais le mieux, la livraison continue par `argocd-image-updater` (chart 0.12.0). Elle s'adresse à un opérateur qui doit configurer, diagnostiquer ou annuler un déploiement automatique d'image.

**Activer le write-back.** Trois annotations suffisent, posées sur l'ApplicationSet : `image-list` déclare les images surveillées, `update-strategy` fixe la détection (chez nous, tag de branche puis épingleage du digest, faute de SemVer), et `write-back-method: git` écrit le nouveau digest dans le dépôt plutôt que de patcher l'objet en cluster. Le write-back par Git est le seul cohérent avec GitOps : l'état désiré vit dans le dépôt, pas dans une mutation que la prochaine synchronisation écraserait.

**Gérer le GITHUB\_TOKEN.** L'updater a besoin de deux droits qu'on confond : lire les digests sur GHCR, et pousser le write-back sur le dépôt. Un token en lecture seule produit le symptôme le plus trompeur, l'image est détectée mais le commit échoue, et le cluster reste sur l'ancien digest sans alerte. Vérifier que le compte du token peut pousser sur `main`.

**Runbook « image non synchronisée ».** Diagnostic dans l'ordre : les logs du contrôleur (`kubectl -n argocd logs deploy/argocd-image-updater`), qui distinguent détection, erreur d'authentification et write-back en échec ; puis l'historique Git sur `kustomization.yaml` pour confirmer que le digest a été écrit ; enfin l'état de l'Application ArgoCD. Cet ordre évite l'erreur classique de déboguer le Pod alors que le digest n'a jamais été écrit.

**Rollback.** Par Git, jamais par mutation directe : on restaure le digest sain par un commit de revert et ArgoCD resynchronise. Un `kubectl rollout undo` ramènerait un état que GitOps ignore et que la prochaine réconciliation écraserait. Mieux vaut suspendre l'updater pendant l'opération, sinon il re-proposera le digest le plus récent.

### 4.2 Analyse personnelle

**Défis rencontrés.** Le défi qui m'a le plus marqué n'est pas un bug, c'est une position : tenir la chaîne de validation, c'est tenir une barrière qui, mal placée dans le temps, bloque toute l'équipe. L'épisode des manifests rejetés m'a appris à dissocier deux gestes que je confondais, mettre au point une règle et la rendre bloquante. J'ai calé mes exclusions dans ma branche pendant que les autres poussaient, puis activé les `required status checks` une fois la mise au point finie. L'ordre compte autant que la règle.

**Forces et faiblesses.** Ma force principale est la rigueur sur la qualité logicielle : je n'ai pas cédé sur le principe d'une CI entièrement verte avant tout merge, même quand l'exigence ralentissait. Ma faiblesse principale est l'estimation du temps d'apprentissage des outils nouveaux. J'ai abordé kubeconform et kube-score en supposant une configuration rapide, et j'ai sous-estimé la profondeur des specs à maîtriser ; le coût s'est lu dans le taux de succès initial. Un outil de validation ne se configure pas en une après-midi, et ce temps doit être budgété au planning.

**Compétences développées.** Avant le projet, je maîtrisais Git, les commandes shell et Linux, et la syntaxe YAML de base. Pendant le projet, j'ai développé la conception de workflows CI industriels, l'analyse statique de manifests Kubernetes qui m'était inconnue, et l'automatisation d'un outil GitOps fondé sur l'écoute d'un registre d'images. Surtout, l'analyse de conformité par Trivy, kube-score et kubeconform est devenue pour moi un standard non négociable : bloquer les dérives avant le déploiement, pas après. C'est la compétence que j'emporte le plus clairement vers une pratique professionnelle.

**Axes d'amélioration personnels.** Quatre axes pour la suite : budgéter le temps

d'apprentissage d'un outil avant de l'intégrer plutôt que de le découvrir sur des runs rouges, pinner les versions dès le premier commit pour qu'un pipeline reste reproductible, poser tôt la lisibilité des releases au lieu de la repousser après le MVP, et documenter les exclusions au fil de l'eau pour qu'une dérogation reste auditable.